

Lab 02 — Hands-on lab: dissect an image, run a registry

Goal: work with tags, layers and digests, then publish an image to a private registry you run yourself — and find out along the way why Podman makes you spell names out in full.

Prerequisites — Lab 01 completed (rootless Podman under WSL, `systemd` active). Port `5000` must be free (`ss -lntp | grep :5000` must return nothing).

Files provided — `files/Dockerfile` (two lines, explained in lab 04; here it only serves to generate images).

Step 1 — Read an image name

```
podman pull nginx:alpine
podman pull alpine
```

Observe Resolved "nginx" as an alias, then Trying to pull `docker.io/library/nginx:alpine...` and the Copying blob lines. For `alpine`, only a bare identifier appears: that image has been on disk since lab 01.

```
podman image inspect --format '{{.RepoTags}}' nginx:alpine
podman image inspect --format '{{.Digest}}' nginx:alpine
```

Observe the contrast: the first command prints `[docker.io/library/nginx:alpine]` — the **full** name, which you never typed — and the second prints `sha256:1f25fedd50aec27413031afb...`.

Explanation. `nginx:alpine` is a readable, movable name; the digest is the content's real, permanent identity. Podman always displays the full name: there is no such thing as a "short name" in its storage, only in your command.

Now try a name that is not in the alias list:

```
grep -c '=' /etc/containers/registries.conf.d/000-shortnames.conf
grep -E '^s*"(\alpine|nginx|eclipse-temurin)"' /etc/containers/registries.conf.d/000-
shortnames.conf
grep unqualified-search-registries /etc/containers/registries.conf
```

Observe that `alpine` and `nginx` have an alias, `eclipse-temurin` does not, and Ubuntu's search list contains only `docker.io` — which is why `podman pull eclipse-temurin:21-jre-alpine` still works on your machine, while it would trigger a prompt on Fedora.

PODMAN

Short names are a convenience for the terminal, not a practice for production. In a Dockerfile or a script, write `docker.io/library/eclipse-temurin:21-jre-alpine`: the result then no longer depends on how the executing machine is configured.

Step 2 — List your images

```
podman images
```

Observe the `REPOSITORY / TAG / IMAGE ID / CREATED / SIZE` columns, with repositories written out in full: `docker.io/library/nginx`, `docker.io/library/alpine`.

```
podman images --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}'
podman images --format '{{.Repository}}:{{.Tag}}'
```

Explanation. Make `--format` a habit: your commands stop depending on display changes, and they become usable in scripts.

Step 3 — The layers, and where the weight goes

```
podman history nginx:alpine --format 'table {{.Size}}\t{{.CreatedBy}}'
```

Observe a single big layer (`50.7MB`, the nginx installation), a few small `COPY` layers holding scripts, an `8.7MB` layer (Alpine) at the very bottom, and many `0B` lines.

Explanation. The `0B` lines are **metadata** instructions: `ENV`, `CMD`, `EXPOSE`, `ENTRYPOINT`, `STOPSIGNAL`. They create no files. Run this command first whenever an image is unexpectedly large: the offending line stands out immediately.

Podman can also show the layers as a tree, together with their source image:

```
podman image tree nginx:alpine
```

Observe the first layer marked `Top Layer of: [docker.io/library/alpine:latest]: nginx:alpine` is **built on** the alpine image you already have — that 8.7 MB layer is stored only once.

```
podman system df
podman system df -v | head -n 12
```

Observe the `SHARED SIZE` column in verbose mode: `8.698MB` for `nginx` and for `alpine` — the same layer, counted in both.

Step 4 — `podman tag` copies nothing

```
podman tag nginx:alpine my-nginx:v1
podman tag nginx:alpine my-nginx:preprod
podman images --format 'table {{.Repository}}\t{{.Tag}}\t{{.ID}}' | grep nginx
```

Observe three lines... all with **the same** `IMAGE ID` — and your two new names carrying the `localhost/` prefix.

```
podman rmi my-nginx:v1
```

Observe the output: only `Untagged: localhost/my-nginx:v1`. No `Deleted:`.

```
podman rmi my-nginx:preprod
```

Observe `Untagged:` again, and nothing more — because `docker.io/library/nginx:alpine` still names the image.

Explanation. A tag is a reference. As long as one name remains, the data remains. That is why "I ran `rmi` and got no disk space back" is such a common complaint — and perfectly normal behaviour. As for `localhost/`: an image you name without a registry belongs to no registry, and Podman says so.

Step 5 — Build two versions of the same image

Copy the provided file into a working folder:

```
mkdir -p ~/labo-docker/02 && cd ~/labo-docker/02
cp <lab-path>/files/Dockerfile .
cat Dockerfile
```

```
podman build -t demo-layers:1.0 .
podman history demo-layers:1.0 --format 'table {{.ID}}\t{{.Size}}\t{{.CreatedBy}}'
```

Observe the lines `STEP 1/2`, `STEP 2/2`, `COMMIT demo-layers:1.0` and `Successfully tagged localhost/demo-layers:1.0`, then three layers: your `RUN` (2.05kB), the base image's `CMD` (0B), and the Alpine file system (8.7MB), marked `<missing>` because that layer belongs to another image.

Change the version, then rebuild on **the same tag**:

```
sed -i 's/version 1/version 2/' Dockerfile
podman build -t demo-layers:1.0 .
podman run --rm demo-layers:1.0 cat /version.txt
```

Observe `version 2`, and a new `IMAGE ID` behind the same tag.

```
podman images --filter dangling=true
```

Observe a `<none> <none>` line showing the **old** `IMAGE ID`: that is the *dangling* image, the one that lost its name. It still occupies 8.7 MB (shared, as it happens).

Explanation. The tag `demo-layers:1.0` **moved** to a new image — exactly the scenario from question 2, and nothing warned you. Remove the leftover by its ID:

```
podman rmi $(podman images --filter dangling=true -q)
```

Step 6 — Run a private registry

A registry is just another container:

```
podman run -d -p 5000:5000 --name lab-registry registry:2
podman ps --filter name=lab-registry
curl -s http://localhost:5000/v2/_catalog
```

Observe `0.0.0.0:5000->5000/tcp` in the `PORTS` column, then `{"repositories":[]}`: the registry is empty and working.

→ WINDOWS / WSL

Port 5000 is published inside the WSL VM, but Windows sees it too: open `http://localhost:5000/v2/_catalog` in your Windows browser. WSL 2 automatically relays ports that Linux listens on to `localhost` on the Windows side (*localhost forwarding*). This is what will let you test the Angular front end from Edge or Chrome in later labs.

Publish your image to it:

```
podman tag demo-layers:1.0 localhost:5000/base/demo:1.0
podman push localhost:5000/base/demo:1.0
```

Observe the failure:

```
Error: ... pinging container registry localhost:5000: Get "https://localhost:5000/v2/":
http: server gave HTTP response to HTTPS client
```

Explanation. Your registry speaks HTTP, and Podman demands HTTPS by default — **even for localhost**, where Docker makes a silent exception. For a test registry, you state that explicitly:

```
podman push --tls-verify=false localhost:5000/base/demo:1.0
curl -s http://localhost:5000/v2/_catalog
curl -s http://localhost:5000/v2/base/demo/tags/list
```

Observe the `Copying blob` lines, `Writing manifest to image destination`, then `{"repositories":["base/demo"]}` and `{"name":"base/demo","tags":["1.0"]}`.

→ SECURITY

The alternative is to declare the registry in `~/.config/containers/registries.conf` (`[[registry]]`, `location = "localhost:5000"`, `insecure = true`). That is convenient on a development workstation and dangerous everywhere else: "insecure" means neither the registry's identity nor its encryption gets verified, so an attacker on the network can swap in a different image. In a company, a registry gets a certificate — end of discussion.

In three commands you have reproduced what your company's CI does. Now check the differential transfer:

```
podman tag demo-layers:1.0 localhost:5000/base/demo:1.1
podman push --tls-verify=false localhost:5000/base/demo:1.1
```

Observe that the same blobs are listed, yet the transfer completes instantly: the registry already has them, so only the manifest gets written.

Step 7 — Pull by digest

Fetch the digest as the registry knows it:

```
curl -sI -H "Accept: application/vnd.oci.image.manifest.v1+json" \
  http://localhost:5000/v2/base/demo/manifests/1.0 | grep -i docker-content-digest
```

Observe a line `Docker-Content-Digest: sha256:239acdd...`. Copy that value.

```
podman rmi localhost:5000/base/demo:1.0
podman pull --tls-verify=false localhost:5000/base/demo@sha256:<paste_here>
podman images --format 'table {{.Repository}}\t{{.Tag}}\t{{.ID}}\t{{.Digest}}' | grep demo
```

Observe that the image comes down, and that `podman image inspect --format '{{.Digest}}' demo-layers:1.0` returns exactly the value you just pasted.

Explanation. Serious deployments use this form because it is **tamper-proof**. Even if someone republishes `base/demo:1.0` with different content, your digest still points at the image you tested.

Step 8 — Carry an image without a network

```
podman save -o /tmp/demo.tar demo-layers:1.0
ls -lh /tmp/demo.tar
tar -tf /tmp/demo.tar | head -n 4
```

Observe an `8.4M` archive containing `<sha256>.tar` files (the layers), a `.json` file (the configuration) and a `manifest.json`: this is the historical `docker-archive` format.

```
podman save --format oci-archive -o /tmp/demo-oci.tar demo-layers:1.0
tar -tf /tmp/demo-oci.tar | head -n 4
```

Observe `blobs/sha256/...` and `index.json` this time: the **OCI** layout, the one every tool (Docker, Podman, skopeo, Kubernetes) can read.

Compare with `export`, which operates on a **container**:

```
podman run -d --name tmpx nginx:alpine
podman export tmpx | podman import - nginx-flat:v1
podman image inspect --format '{{json .Config.Cmd}}' nginx-flat:v1
podman run --rm nginx-flat:v1
```

Observe `null`, then the error `crun: cannot find `` in $PATH`: the imported image **has no idea what to launch**.

Explanation. Remember the rule: `save` / `load` for an image; `export` / `import` never for deployment.

```
podman rmi demo-layers:1.0 localhost:5000/base/demo:1.1
podman load -i /tmp/demo.tar
```

Observe `Loaded image: localhost/demo-layers:1.0`: the tag travelled with the archive.

Clean-up

```
podman rm -f -t 0 tmpx lab-registry
podman rmi nginx-flat:v1 demo-layers:1.0 registry:2
podman images --format '{{.Repository}}:{{.Tag}}' | grep -E 'demo|flat|registry'
rm -f /tmp/demo.tar /tmp/demo-oci.tar
```

The image you pulled by digest in step 7 may still be around:

```
podman images --format 'table {{.ID}}\t{{.Repository}}' | grep localhost:5000
podman rmi <ID>
```

Observe that only `docker.io/library/alpine` and `docker.io/library/nginx:alpine` remain — keep those for the next labs.

What you must be able to state now

- A tag is a movable reference; the digest identifies content. Podman stores and displays full names, and prefixes your own with `localhost/`.
- `podman history` and `podman image tree` reveal where an image's weight goes and what it shares.
- `podman tag` duplicates nothing; `podman rmi` removes a name before it ever removes data.
- A `push` transfers only the layers the registry is missing.
- A registry is a plain HTTP service you can start with one command — but Podman demands TLS, unless you pass `--tls-verify=false` explicitly.
- `export` / `import` destroys the image configuration; `save` / `load` preserves it, in `docker-archive` or `oci-archive` format.